

RNGD NPU vs NVIDIA GPU

코드 생성 모델 서빙 벤치마크

같은 모델 · 같은 조건으로 Furiosa RNGD 와 NVIDIA A6000 에서 5 개 테스트를 측정 · 비교

핵심 결론

8B 모델 단일 속도는 NPU 가 30% 빠르다

Llama-3.1-8B 단일 출력 NPU 54.5 vs GPU 41.9 tok/s · 동시 사용자 32 명 부근에서 GPU 역전 · 코드 정확도 (SWE-bench) 는 두 디바이스 모두 0% — 정확도는 하드웨어가 아닌 모델 크기가 결정

측정 테스트

5 종

속도 · 동시성 · serve 옵션 · SWE-bench · 임베딩

측정 디바이스

2 종

Furiosa RNGD NPU · NVIDIA A6000 GPU

비교 모델

2 종

Qwen2.5-0.5B(smoke) · Llama-3.1-8B

무엇을 , 어떤 기준으로 검증했나

RNGD NPU 와 A6000 GPU 에서 코드 생성 모델 서빙을 4 개 축으로 측정 · 비교

검증 목표

Furiosa RNGD NPU 가 코드 생성 워크로드에서 NVIDIA GPU 대비 어느 지점에 강하고 어느 지점에 약한지를 , 같은 모델 · 같은 조건의 정량 측정으로 가려낸다 .

01

토큰 생성 속도

속도

단일 요청에서 첫 토큰 지연 (TTFT) 과
초당 생성 토큰 수 (tok/s) 를
NPU·GPU 각각 측정

해당 테스트 · 테스트 1

02

동시 접속 확장성

동시성 · 배치

동시 요청을 1→128 로 늘리며 합산
처리량을 측정 — NPU·GPU 의
교차점을 찾는다

해당 테스트 · 테스트 2·3

03

SWE-bench

정확도

실제 GitHub 이슈를 코드 패치로
해결하는 정확도 — 하드웨어 의존성
검증

해당 테스트 · 테스트 4

04

서빙 · 자동화

환경

furiosa-llm·vLLM 서빙 구성과 모델
추가만으로 도는 측정 파이프라인 구축

해당 테스트 · 테스트 5 · 전반

측정 환경 — 같은 클라이언트, 다른 디바이스

측정 도구는 OpenAI 호환 API 만 호출 → 서버 계층만 바뀌 NPU·GPU 를 같은 코드로 측정

측정 클라이언트 (공통)

httpx 비동기 스트리밍
/v1/chat/completions 호출
tps·sweep·swebench 코드 100% 공유



NPU 경로

furiosa-llm serve

RNGD NPU · prebuilt 아티팩트 실행

GPU 경로

vllm serve

A6000 GPU · 원본 HF 모델 실행

서빙

OpenAI 호환 API

furiosa-llm·vLLM 모두 동일한 /v1 endpoint 를 제공
→ 측정 도구가 디바이스를 구분하지 않는다.

측정

스트리밍 클라이언트

응답을 토큰 단위 스트림으로 받아 TTFT· 토큰 간 지연을
실측. 동시 요청은 비동기로 발생.

채점

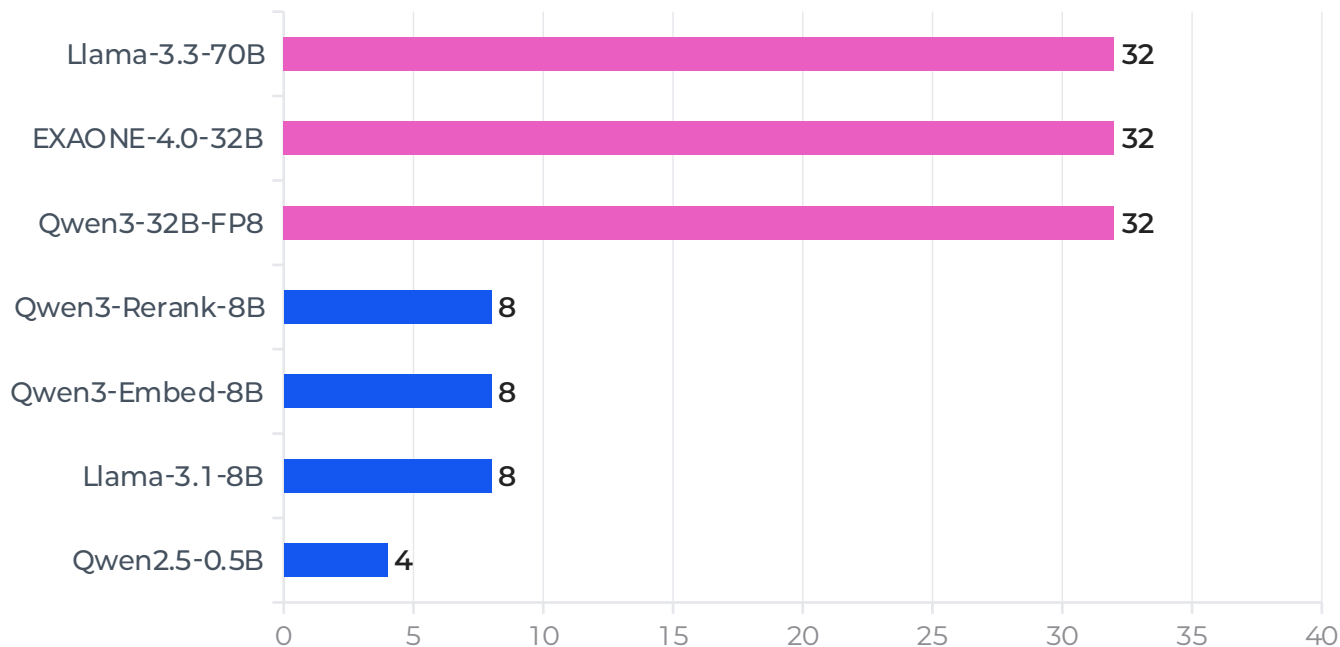
Docker harness

SWE-bench 채점에만 Docker 사용 — NPU·GPU
동일하게 인스턴스별 격리 컨테이너에서 테스트 실행.

7종 중 4종만 RNGD 2-카드에서 측정 가능

prebuilt 아티팩트의 tensor-parallel 크기가 필요 NPU 수를 고정한다

모델별 필요 PE (RNGD 1 카드 = 8 PE)



파랑 = 2 카드 (16 PE) 로 측정 가능 · 분홍 = 4 카드 필요 → 측정 불가

측정 대상 4 종

Qwen2.5-0.5B · Llama-3.1-8B · Qwen3-Embedding-8B
· Qwen3-Reranker-8B — 모두 1 카드 (≤8 PE)

측정 제외 3 종

Qwen3-32B-FP8 · EXAONE-4.0-32B-FP8 · Llama-3.3-70B — 아티팩트가 tp=32(4 카드) 로 컴파일

NPU↔GPU 비교 모델

생성 모델 Qwen2.5-0.5B·Llama-3.1-8B 2 종을 GPU 에서도
동일 측정 — 이 텍스트의 비교 대상

공정 비교 — 엔진 외 모든 변수를 일치

서빙 엔진 (furiosa-llm vs vLLM) 만 다르고 , 나머지 측정 조건은 NPU·GPU 동일

조건 정렬 표

항목	NPU (RNGD)	GPU (A6000)
측정 모델	Qwen2.5-0.5B · Llama-3.1-8B	동일 (원본 HF)
정밀도	bf16 (양자화 없음)	bf16 (양자화 없음)
컨텍스트 — Qwen	4,096 (artifact 한도)	4,096 (--max-model-len)
컨텍스트 — Llama	32,768 (artifact 한도)	32,768 (--max-model-len)
prefix caching	ON	ON (--enable-prefix-caching)
sweep 그리드	동시성 1-128 × prompt 256/1K/4K	동일
SWE-bench	Lite oracle · single-shot · 50 건	동일

정렬이 실제로 됐다는 증거

sweep 에서 prompt=4096 셀은 NPU·GPU 모두 전건 실패한다 (prompt 4096 + max_tokens 256 > 4096 한도). 양쪽이 똑같이 실패했다는 것이 4K 컨텍스트가 동일하게 적용됐다는 직접 증거다 .

주의 — 성능 비교만

A6000 과 RNGD 는 가격 · 소비전력 제품군이 다르다 . 이 텍은 동일 SW 조건의 처리 성능만 비교하며 , 전력당 · 비용당 성능 (TCO) 은 별도 측정 과제다 .

처음 GPU 측정은 컨텍스트가 8 배 (32K) 차이 나 불공정했다 → **NPU artifact 한도 (4K·32K) 에 맞춰 재측정한 결과가 이 텍의 데이터다 .**

측정 파이프라인 — 모델 추가만으로 자동화

configs/models.yaml 에 모델을 등록하면 서버 기동부터 리포트까지 자동 실행 (NPU·GPU 동일)



측정 코드는 NPU·GPU 100% 공유 — server 계층만 다름

한 모델의 서버를 띄워 테스트 1·2·4 를 측정하고, 테스트 3(memswep)은 serve 옵션을 바꿔 서버를 재기동하며 측정한다. NPU 는 furiosa-llm, GPU 는 vLLM 으로 서버만 갈아끼울 뿐 tps·sweep·swebench·embed 측정 코드는 같다 — 그래서 두 디바이스 결과를 같은 표에 올릴 수 있다. 새 모델은 models.yaml 에 한 줄 추가하면 전 과정이 자동 반복된다.

측정 테스트 5 종 한눈에

각 모델 · 각 디바이스에 대해 아래 5 개 테스트를 순차 수행 — 이후 슬라이드는 테스트 번호 순

테스트 1	테스트 2	테스트 3	테스트 4	테스트 5
토큰 생성 속도	동시성 스케일링	serve 옵션	SWE-bench	임베딩 · 리랭커
단일 요청 스트리밍으로 첫 토큰 지연 · 생성 속도 측정	동시 요청을 1→128 로 늘리며 처리량 · 지연 변화 측정	serve 인자를 바꿔가며 처리량 영향 측정 (memsweep)	실제 GitHub 이슈를 코드 패치로 해결, 테스트 통과 채점	검색 보조 모델의 배치별 처리량 측정
지표 TTFT · tok/s	지표 합산 TPS · 실패율	지표 조합별 합산 TPS	지표 resolved %	지표 inputs/s

테스트 1 — 토큰 생성 속도 (NPU vs GPU)

단일 요청 스트리밍 — 8B 는 NPU 가, 0.5B 는 GPU 가 빠르다

무엇을

동시 사용자 1 명일 때 체감 속도 — 코드 자동완성 · 대화형 보조의 핵심 지표 .

입력

고정 프롬프트를 /v1/chat/completions 에 스트리밍 요청 .
max_tokens 256, 워밍업 5 + 측정 50 회 .

출력 · 분석

요청별 TTFT·ITL(토큰 간 간격) · 출력 tok/s 를 NPU·GPU 각각 p50 으로 집계 .

결과

Llama-3.1-8B-Instruct

코드 생성 후보

NPU +30%

NPU

54.5 tok/s

단일 요청 출력 토큰 처리량

GPU

41.9 tok/s

Qwen2.5-0.5B-Instruct

smoke · 검증용

GPU 3.0×

NPU

84.5 tok/s

단일 요청 출력 토큰 처리량

GPU

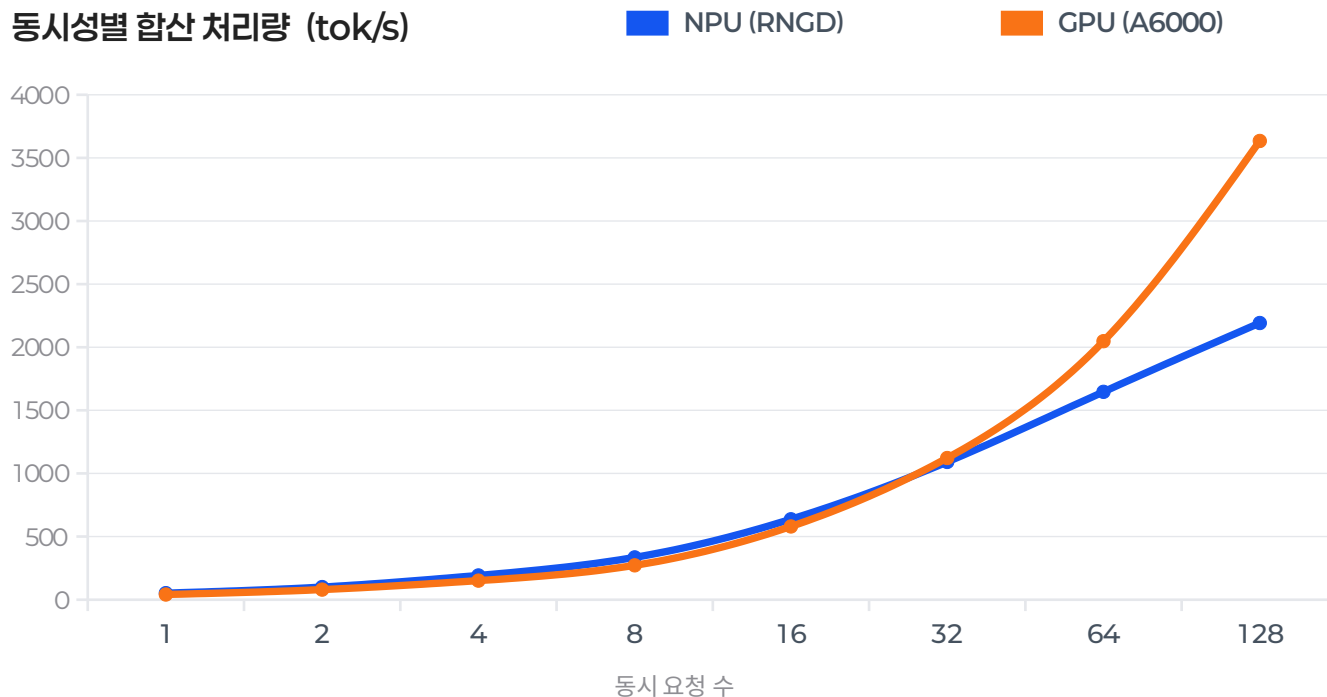
249 tok/s

왜 갈리나 RNGD 는 수 B 이상 모델에서 연산 파이프라인 효율이 살아난다 — 8B 는 NPU 우위 , 0.5B 는 너무 작아 메모리 대역폭 싸움이 되어 GPU 우위 .

테스트 2 — 동시성 스케일링 (NPU vs GPU)

동시 요청 1→128 — c16 까지 NPU 우세, c32 교차, c128 은 GPU 가 66% 앞선다

동시성별 합산 처리량 (tok/s)



c1-c16 NPU 우세 구간

동시 16 명까지 NPU 가 빠르다 (c8: 335 vs 272 tok/s, +23%). 소수 사용자 코드 어시스턴트 · IDE 플러그인에 적합.

c32 교차점

동시 32 명 부근에서 역전 (1091 vs 1122). 이 지점이 NPU-GPU 선택 기준선.

c64-c128 GPU 우세 구간

GPU 의 배치 처리량이 앞선다 (c128: 3634 vs 2192, +66%). 대규모 동시 트래픽 공용 API 에 적합.

테스트 3 — serve 옵션 튜닝 (NPU vs GPU)

서버 옵션을 한 축씩 바꿔 (OFAT) 합산 처리량 영향 측정

무엇을

KV cache·배치 관련 serve 옵션이 처리량에 주는 영향을 NPU·GPU 각각 측정.

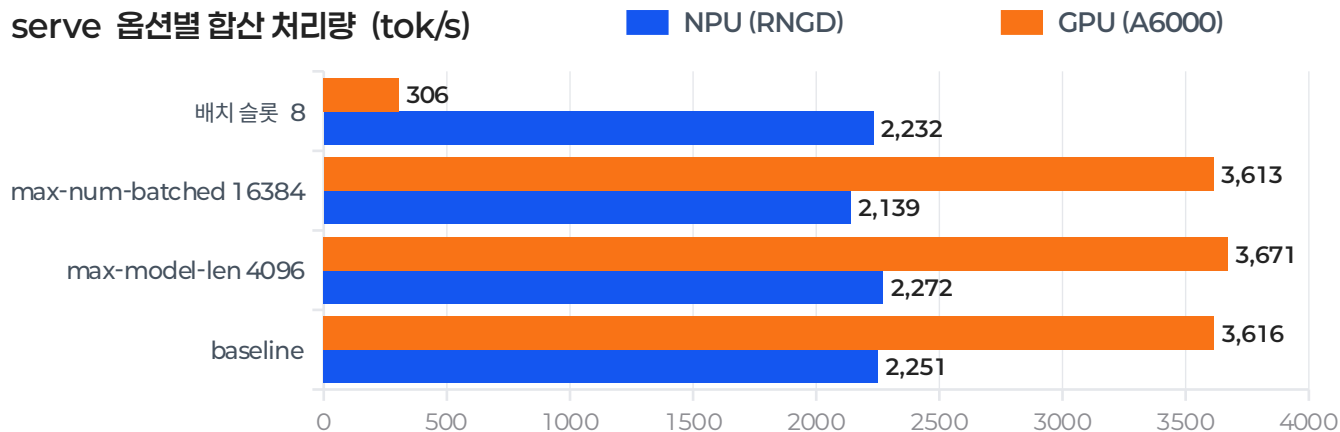
입력

baseline + max-model-len·배치 슬롯·max-num-batched-tokens 한 축씩 변경, 조합마다 서버 재기동.

출력·분석

조합별 합산 처리량 비교 → 어떤 옵션이 유의미한지, 디바이스별 민감도 차이 판단.

결과



정상 옵션은 튜닝 효과 작음

max-model-len·max-num-batched-tokens 를 바꿔도 처리량 변동은 양쪽 모두 수 % 이내. 기본값 서빙으로 충분하다.

배치 슬롯 축소 — 결정적 차이

슬롯을 8로 줄이면 GPU(vLLM)는 처리량이 92% 붕괴 (3616→306). NPU(furiosa-llm)는 -1%로 거의 무영향.

운영 시사점

GPU는 max-num-seqs를 충분히 크게 줘야 한다. NPU는 이 옵션에 둔감 — 튜닝 부담이 작다.

테스트 4 — SWE-bench ①: 정의 & 종류

GitHub 이슈를 패치로 해결하고 실제 repo 테스트 통과 여부로 채점하는 벤치마크

채점 방식

1

입력

GitHub 이슈 본문 + 해당 시점 repo 코드



2

모델 출력

이슈를 해결하는 unified diff (패치)



3

적용

Docker 컨테이너에서 repo 에 패치 적용



4

채점

테스트 실행 — FAIL→PASS 통과 시 resolved

데이터셋 종류

SWE-bench (full)

2,294 건 · 원본 12 개 repo

SWE-bench Lite

300 건 · 저비용 평가용 subset

SWE-bench Verified

500 건 · 사람 검증, 신뢰도 최상

SWE-bench Multimodal

517 건 · 시각 요소 포함 (JS/UI)

본 측정

SWE-bench Lite · oracle · single-shot

Lite 300 건 중 12 개 repo(astropy·django·matplotlib·sympy 등) 에 고르게 50 건 추출.
context 는 oracle(정답 파일 제공), 호출은 single-shot(1 회 생성) — 에이전트 반복 없이 순수 코드 편집 능력을 본다. NPU·GPU 동일 구성.

테스트 4 — SWE-bench ②: 시작 가이드

Docker 기반 채점 — 인스턴스별 격리 컨테이너에서 repo 테스트 실행

① 설치

```
$ git clone https://github.com/SWE-bench/SWE-bench.git
$ cd SWE-bench && pip install -e .
$ docker --version # Docker 동작 확인
```

② 채점 실행

```
$ python -m swebench.harness.run_evaluation \
  --dataset_name princeton-nlp/SWE-bench_Lite \
  --predictions_path predictions.jsonl \
  --max_workers 8 \
  --namespace swebench \
  --run_id my_run
# --namespace swebench → prebuilt 이미지 pull
# 결과 : <model>.<run_id>.json (resolved 카운트)
```

요구사항

- x86_64 Linux · Docker
- 디스크 수심 ~120GB (인스턴스 이미지)
- Python 3.10+ · swebench 패키지

예측 파일 (predictions.jsonl)

한 줄 = 한 인스턴스 . 모델이 만든 패치를 model_patch 에 담는다 :

```
{"instance_id": "astropy____"
 "model_name_or_path": "8B",
 "model_patch": "diff --git ..."}

```

테스트 4 — SWE-bench ③: 측정 방법

oracle 프롬프트를 로컬 서버 (furiosa-llm/vLLM) 에 보내 diff 를 받고 harness 로 채점

입력

oracle 프롬프트

이슈 설명 + 수정 대상 파일 코드

```
<issue>
separability_matrix 가
nested CompoundModel 에서
분리성을 잘못 계산함
</issue>

[start of separable.py]
def _separable(transform):
    ...
```

출력

모델 생성 diff

Llama-3.1-8B single-shot 응답

```
--- a/.../separable.py
+++ b/.../separable.py
@@ -304,6 +304,10 @@
     elif isinstance(
         transform, Compound):
+   + if isinstance(
+   +   transform.left,...):
+   +   sepleft =
+   +   _separable(...)
```

채점

Docker harness

패치 적용 후 테스트 실행

resolved

패치 적용 + 테스트 통과

unresolved

적용됐으나 테스트 미통과

적용실패

malformed diff — 적용 불가

테스트 4 — SWE-bench ④: 결과 (NPU vs GPU)

두 디바이스 모두 resolved 0% — 같은 모델은 디바이스가 달라도 같은 정확도

결론

NPU·GPU 4 개 조합 전부 resolved 0 — 정확도는 디바이스가 아니라 모델 크기 · 방식이 결정한다

채점 결과 — 4 개 조합

모델 · 디바이스	resolved	미해결	적용실패	평가건수
Llama-3.1-8B · NPU	0	14	36	50
Llama-3.1-8B · GPU	0	18	29	47
Qwen2.5-0.5B · NPU	0	1	2	3
Qwen2.5-0.5B · GPU	0	1	2	3

미해결 = 적용됐으나 테스트 실패 · 적용실패 = malformed diff. Qwen 은 4K 컨텍스트라 47 건이 평가 불가 (제외) → 3 건만 채점.

resolved 0 — 모델 한계

8B·0.5B 를 single-shot 으로 돌리면 어느 하드웨어에서도 SWE-bench Lite 를 못 푼다.

적용실패율 60~70%

NPU Llama 36/50, GPU Llama 29/47 — 모델이 구조적으로 올바른 diff 를 잘 못 만든다.

디바이스 독립 확인

같은 모델의 NPU·GPU 결과가 거의 일치 → NPU 전환에 따른 정확도 손해는 없다.

테스트 4 — SWE-bench ⑤: 왜 0% 인가

single-shot 8B 의 한계 — 같은 방식이면 GPU·프런티어 모델도 한 자릿수

결론

정확도를 가르는 건 하드웨어가 아니라 모델 크기 · 스케폴드 — 개선은 32B/70B 또는 에이전트로만

Llama-3.1-8B 두 실패 유형 — 모두 모델 역량 문제

적용실패 (NPU 36 · GPU 29)

모델이 diff 의 @@ hunk 헤더 줄 수를 잘못 계산 → patch 도구가 거부. 8B 가 정확한 unified diff 포맷을 못 맞춘다.

미해결 (NPU 14 · GPU 18)

문법이 맞는 패치가 적용됐으나 테스트 미통과 — 수정 위치 · 내용이 틀림. 단발 8B 코드 추론의 한계.

공개 SWE-bench 결과 — 단발 생성 (에이전트 없음)

모델 (규모)	벤치 · N	resolved
Llama-3.1-8B (본 측정 · 8B)	Lite · 50	0 %
GPT-4 (프런티어)	full · 2294	1.7 %
Claude 2 (프런티어)	full · 2294	4.8 %
Llama-3.3-70B (70B)	Verified · 500	5.4 %

핵심 같은 single-shot 이면 8B 보다 큰 GPT-4·Claude 2·70B 도 1.7~5.4%. 정확도는 더 큰 모델 (4 카드 32B·70B) 이나 에이전트 스케폴드로만 오른다.

테스트 5 — 임베딩 · 리랭커

검색 보조 모델 (Qwen3-Embedding-8B · Qwen3-Reranker-8B) 의 처리량 측정 — NPU 단독

무엇을

임베딩 / 리랭킹 처리량 — SWE-bench 검색 보조 (RAG) 에 쓰이는 모델 .

입력

/v1/embeddings/v1/rerank 에 batch {1·4·16·64} 개 입력 .

출력 · 분석

초당 처리 건수 (inputs/s), 배치 크기별 효율 .

결과

Qwen3-Embedding-8B

1.17 inputs/s

batch 1·16·64 모두 동일 — 배치 이득 없음

Qwen3-Reranker-8B

1.17 pairs/s

쿼리당 100 문서 ≈ 85 초 — 항목당 약 0.85 초

처리량 이상 — 별도 점검 필요

두 모델 모두 항목당 약 0.85 초 — 8B 생성 모델의 단일 prefill(수십 ms) 대비 수십 배 느리다 . batch size 를 키워도 처리량이 1.17/s 로 고정돼 배치가 동작하지 않는다 . prebuilt 아티팩트가 단일 배치 (batch=1) 로 컴파일된 것으로 추정 — 대량 검색에 부적합하며 배칭 지원 아티팩트 재컴파일이 후속 과제 . GPU 측은 미측정 (embedding 모델 보류) .

측정 모델 노트 — Qwen2.5-0.5B-Instruct

속도는 GPU 가 압도하지만 컨텍스트 4,096 한계로 코드 작업엔 부적합 — smoke 전용

smoke

자동화 파이프라인 (서빙→측정→채점) 이 정상 동작하는지 빠르게 확인하는 검증용 모델 — models.yaml 에 role: smoke 로 지정된 , 코드 생성 후보가 아닌 모델 .

속도 — 0.5B 는 GPU 가 압도

249 / 84

단일 출력 tok/s (GPU / NPU) — 작은 모델은 메모리 대역폭 싸움 → GPU 3 배 .

11.4K / 4.1K

동시성 c128 합산 tok/s (GPU / NPU) — 전 구간 GPU 우세 .

13 / 31 ms

TTFT p50 (GPU / NPU) — 첫 토큰도 GPU 가 빠르다 .

한계 — 컨텍스트 4,096 토큰

4,096

max_context_len — NPU prebuilt 아티팩트의 고정 버킷 . GPU 도 동일하게 맞춤 .

0 / 3

SWE-bench — oracle 프롬프트 47 건이 4096 초과로 평가불가 , 3 건만 채점 .

sweep 4096

prompt=4096 셀은 NPU·GPU 모두 전 동시성 실패 — 버킷 초과 .

시사점 smoke·단문 고처리량 데모엔 적합 · 코드 생성 · 긴 컨텍스트 작업엔 부적합 — 실사용하려면 아티팩트를 더 큰 컨텍스트 버킷으로 재컴파일해야 한다 .

종합 — NPU 와 GPU 는 성격이 다른 디바이스

RNGD = 지연 (latency) 최적화 · A6000 = 처리량 (throughput) 최적화

RNGD NPU
지연 (latency) 머신

- 8B 단일 출력 속도 +30% (54.5 vs 41.9 tok/s)
- 토큰 간 지연 (ITL)· 첫 토큰 (TTFT) 모두 짧음
- 동시성 c16 까지 합산 처리량 우세
- serve 옵션 (배치 슬롯) 에 둔감 — 튜닝 부담 작음

NVIDIA A6000 GPU
처리량 (throughput) 머신

- 동시성 c128 합산 처리량 +66% (3634 vs 2192)
- 0.5B 같은 소형 모델 전 구간 우세
- 동시 32 명 이상 대규모 트래픽에 강함
- max-num-seqs 충분히 키워야 처리량 유지

워크로드별 권장 디바이스

워크로드	권장	근거
IDE 자동완성 · 소수 사용자 코드 어시스턴트 (동시 ≤ 16)	RNGD NPU	8B 단일 +30%, c16 까지 처리량 우세
대화형 챗봇 — 낮은 지연 우선	RNGD NPU	TTFT·ITL 모두 NPU 가 짧음
대규모 공용 API (동시 ≥ 32)	A6000 GPU	c32 이후 처리량 역전 , c128 +66%
코드 정확도가 핵심	디바이스 무관	8B 는 둘 다 0% — 32B/70B· 에이전트 필요

결론 — 8B 코드 서빙, 동시성으로 디바이스를 고른다

소수 ~ 중간 규모는 RNGD NPU, 대규모 동시 트래픽은 A6000 GPU

최종 결론

8B 급 코드 생성 모델을 동시 16~32 명 이하로 서빙하면 RNGD NPU 가 GPU 보다 빠르고 안정적이다. 동시 사용자가 수십 명을 넘으면 GPU 의 배치 처리량이 앞선다. 코드 정확도는 두 디바이스가 동일 — NPU 전환에 따른 정확도 손해는 없다.

채택

소수 사용자 = RNGD NPU

8B 단일 속도 +30%, c16 까지 처리량 우세. 코드 자동완성·대화형 보조에 적합하며 serve 옵션은 기본값으로 충분.

분담

대규모 트래픽 = A6000 GPU

동시 32 명 이상에서 GPU 처리량이 역전 (c128 +66%).
공용 API 는 GPU, 저지연 구간은 NPU 로 분담 가능.

과제

정확도·임베딩 후속 측정

정확도는 4 카드 32B·70B 측정 필요. NPU 임베딩 1.17/s 이상 현상은 배치 아티팩트 재컴파일로 점검.